Daffodil International University
Department of Software Engineering

System Design Document

Unit 6: Chat & Notifications

TrustBazaar: A Verified Peer-to-Peer Auction and Marketplace Platform for Used Goods

S. M. Nihal Ahmed (232-35-002)
Afrim Hossen Khan (232-35-076)
Sabikun Nahar Sinthia (232-35-475)

Status	Draft for team review
Version	0.1, 18 October 2026
Implements	SRS v2.0 (`docs/srs.tex`)
Depends on	Unit 1 (Identity & Access), Unit 2 (Order, Payment & Escrow), Unit 3 (Auctions, Listings & Search); events of Units 1 to 5
Database schema	`docs/design/sql/06_chat_notifications.sql`, tested on PostgreSQL (69 checks and concurrency tests, Appendix B)

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Requirements covered . . . 3

2 Unit Overview **4**
2.1 Responsibilities . . . 4
2.2 Components . . . 4

3 Interfaces **5**
3.1 HTTP API . . . 5
3.2 Live channel . . . 5
3.3 Ports provided . . . 6
3.4 Events consumed . . . 6
3.5 Ports required . . . 7

4 Data Design **7**
4.1 Tables and rules the database enforces . . . 8

5 Chat **8**
5.1 Threads . . . 8
5.2 Sending and delivery . . . 8
5.3 Text and images . . . 9
5.4 Detecting off-platform payment and contact . . . 9
5.5 Blocking . . . 9
5.6 Reports, holds and staff access . . . 10
5.7 Retention and closed accounts . . . 10

6 Notifications **10**
6.1 From event to notice . . . 10
6.2 Channels . . . 11
6.3 Nothing secret in a notice . . . 11
6.4 Chat e-mails . . . 12
6.5 Auctions . . . 12
6.6 Deadline reminders . . . 12

7 Scheduled Jobs **12**

8 Performance and Scaling **12**

9 Security and Privacy **13**

10 Parameters **13**

11 Testing **13**

12 Notes for Integration **14**

A Traceability **15**

B Database Schema and Tests **15**

B.1 Test results 15

B.2 Schema 17

1 Introduction

1.1 Purpose

This document is the sixth unit of TrustBazaar's System Design Document. It covers the two ways the platform talks to people:

- **chat**: one thread per listing and prospective buyer, delivered in real time, with warnings when a message tries to move payment or contact off the platform;
- **notifications**: what every other unit's events mean for the people involved, delivered in the app, by e-mail and, for security notices only, by SMS, including the deadline reminders.

Chat matters to trust because one of the defining frauds of the market studied for this project starts there: a fake seller asks for an advance payment and disappears (Supplementary Report, Section 3). The design cannot stop people from typing a phone number, but it can make sure both parties see, on that very message, that payments outside TrustBazaar are not protected (FR-CHT-07).

The unit follows conventions DC-1 to DC-14. It implements ports that earlier units already call: Unit 1's `NotificationPort` for OTPs and security notices, and a new `BlockPort` that Units 2 and 3 check before a purchase or a bid.

1.2 Scope

In scope:

- threads, messages, images, delivered and read marks, reconnection;
- detection of off-platform payment and contact, and of login details (FR-GAM-20);
- blocking, reporting a message, holds for disputes and reports, staff access to threads, retention;
- templates in Bangla and English, the channel rules, e-mail and SMS adapters, preferences, the notification history;
- auction watching and ending-soon notices, deadline reminders.

Out of scope, owned by other units:

- the live auction channel and the real-time outbid message (Unit 3; this unit keeps the outbid history);
- OTP generation, checking and rate limits (Unit 1; this unit only sends the SMS);
- the report queue, sanctions and disputes (Unit 7, which places holds on threads through this unit);
- the chat and notification screens (Unit 8).

1.3 Requirements covered

- **Functional:** FR-CHT-01 to FR-CHT-10; FR-NTF-01 to FR-NTF-08; FR-GAM-20; FR-TNS-01 and FR-TNS-09 (reports from chat, the reporter's notice); FR-PRV-04 (chat on account closure).
- **Security:** SEC-SES-07 (chat rooms); SEC-ABU-01 (chat limit); SEC-DAT-07 and SEC-DAT-08 (nothing secret in messages sent; image metadata); SEC-INP-04; SEC-LOG-01 (staff reads of chats); SEC-AUTH-09 (OTP delivery).
- **Other:** IF-SW-05, IF-SW-06, IF-COM-02, IF-COM-03; NFR-PERF-04; NFR-SCL-01; DR (chat retention).

2 Unit Overview

2.1 Responsibilities

1. Deliver every chat message exactly once, in order, within a second to an online recipient, and on reconnection to an offline one.
2. Warn both parties whenever a message looks like a phone number, an account number, a link, an e-mail address, an advance-payment request or login details, without blocking the message.
3. Let a user block another, and let staff read a thread only for a case that concerns it.
4. Turn each event of the other units into the right notices, once, on the right channels, in the user's language, and never with anything secret in them.
5. Remind people of their deadlines in time, and never after the deadline has passed.

2.2 Components

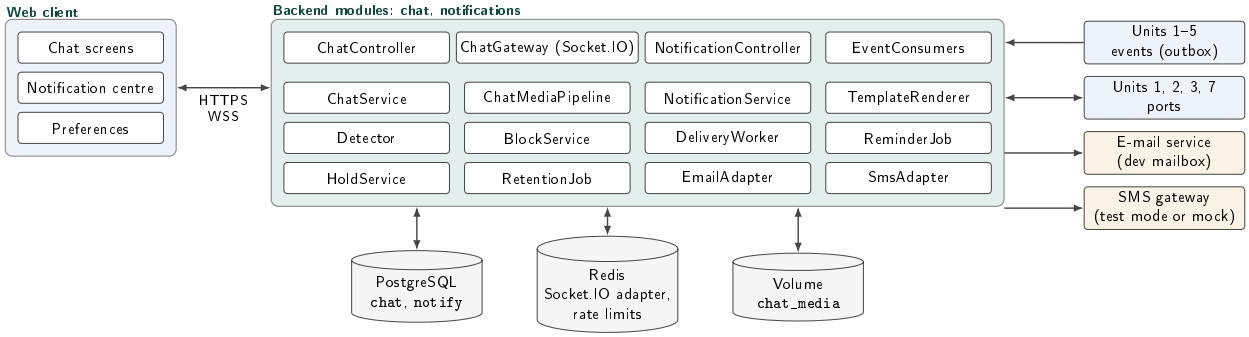


Figure 1: Components of the Chat & Notifications unit. Messages between backend instances pass through the Socket.IO Redis adapter; e-mail and SMS leave only through the two adapters.

Component	Responsibility
ChatService	Opens threads, posts messages through `chat.post`, applies the chat rate limit (CP-35), syncs history (Section 5).
ChatGateway	Socket.IO events for chat and notifications on the authenticated connection; rooms `user:{id}` only (SEC-SES-07).
ChatMediaPipeline	Checks, strips and re-encodes chat images, stores them privately, issues signed links to participants (Section 5.3).
Detector	The rules of Section 5.4; the database function `chat.detect` is the implementation used on every message.
BlockService	Blocks and unblocks; implements `BlockPort` for Units 2 and 3 (Section 5.5).
HoldService, RetentionJob	Holds for disputes and reports; staff reads; the retention purge (Section 5.6).
EventConsumers	One idempotent consumer per event type of Units 1 to 5; each maps an event to notices and reminders (Section 3.4).
NotificationService, TemplateRenderer	Creates notifications through `notify.create`; renders the template in the recipient's language at read or send time.
DeliveryWorker, adapters	Sends pending e-mail and SMS with retries; the adapters are the only code that talks to the providers (Section 6.2).
ReminderJob	Claims due reminders, checks with the owning unit that the deadline still applies, and sends them (Section 6.6).

3 Interfaces

3.1 HTTP API

All paths are under `/api/v1`; policies and rate limits are those of Unit 1. Another user is shown only by display name and badge (FR-ACC-10).

Method	Path	Policy	Purpose (requirement)
Chat			
`POST`	`/listings/{id}/threads`	SignedIn	Open (or find) my thread with the seller (FR-CHT-01)
`GET`	`/threads`	SignedIn	My threads, newest first, with unread counts
`GET`	`/threads/{id}/messages?after=`	Participant	Messages after a sequence number; used to resync (FR-CHT-02, IF-COM-02)
`POST`	`/threads/{id}/messages`	Participant	Send text, with the client's message id (FR-CHT-02)
`POST`	`/threads/{id}/images`	Participant	Send an image (FR-CHT-03)
`POST`	`/threads/{id}/marks`	Participant	Delivered and read up to (FR-CHT-05)
`POST`	`/threads/{id}/reports`	Participant	Report the thread or a message (FR-CHT-06)
`POST`, `DELETE`	`/blocks/{userId}`	SignedIn	Block or unblock a user (FR-CHT-08)
Notifications			
`GET`	`/notifications`	SignedIn	History, newest first; cursor (FR-NTF-04)
`POST`	`/notifications/read`	SignedIn	Mark some or all as read
`GET, PUT`	`/me/notification-preferences`	SignedIn	Language; transactional e-mails on or off (FR-NTF-05)
`POST`, `DELETE`	`/listings/{id}/watch`	SignedIn	Watch an auction (FR-NTF-02)
Providers and staff			
`POST`	`/webhooks/sms/{provider}`, `/webhooks/email/{provider}`	Provider signature	Delivery receipts and bounces (IF-COM-03)
`GET`	`/staff/cases/{type}/{caseId}/threads/{id}`	Staff(operator), case	Read a thread for a case that holds it; logged (FR-CHT-10)
`POST`, `DELETE`	`/staff/chat/detection-terms`	Staff(operator)	Maintain the detection terms
`POST`, `PATCH`	`/admin/notification-templates`	Staff(admin)	New template versions

3.2 Live channel

The Socket.IO connection authenticated by Unit 1 (SEC-SES-07) joins only the room `user:{id}` for chat and notifications; there are no thread rooms, so a user can never be subscribed to a thread they are not in.

Message	Content and use
`chat.message`	To both participants after commit: thread, sequence number, sender, kind, text or a signed image link, flags. The client applies messages in sequence order and, on a gap, fetches the missing ones.
`chat.ack (client to server)`	"Delivered up to" as soon as a message arrives, "read up to" when it is shown.
`chat.marks`	To the sender: the other side's new delivered and read marks.

Message	Content and use
`notification.new`	The new notification's id, title in the user's language and the unread count.

3.3 Ports provided

```
interface NotificationPort {                          // used by Unit 1 (Unit 1 Section 2)
  sendSms(phone: string, purpose: 'otp' | 'security_notice', template: string, vars: object): Promise<void>;
  sendEmail(address: string, template: string, vars: object): Promise<void>;   // links (verification, reset)
}
interface BlockPort {                                 // used by Units 2 and 3 (FR-CHT-08)
  assertNotBlocked(sellerId: string, buyerId: string): Promise<void>;          // throws 'blocked'
}
interface ChatCommands {                              // used by Unit 7
  hold(threadId: string, caseType: 'dispute' | 'report', caseId: string): Promise<void>;
  release(threadId: string, caseType: 'dispute' | 'report', caseId: string): Promise<void>;
  threadFor(listingId: string, buyerId: string): Promise<string | null>;      // the thread of an order
}
interface Notifier {                                  // used by Unit 7 (decisions, FR-TNS-09)
  notify(userId: string, template: string, params: object, dedupeKey: string): Promise<void>;
}
```

3.4 Events consumed

Each event of another unit maps to notices for the people concerned. Every consumer is idempotent: the dedupe key is the event's id, the recipient and the template, so an event delivered twice creates one notice (DC-6).

Events	Notice to, channels	Reminders scheduled or cancelled
`identity.signin.new_device`, `.password.changed`, `.contact.changed`, `.mfa.*`; `payments.payout_account.changed`	The user; app, e-mail, SMS; cannot be switched off (FR-NTF-06)	—
`identity.kyc.verified`, `.rejected`, `.account.suspended`, `.reinstated`	The user; app, e-mail (FR-NTF-03, FR-TNS-03)	—
`orders.order.created` (auction or second chance)	Buyer	payment window
`orders.order.paid`	Buyer and seller; app, e-mail	cancel payment window; schedule meetup, pickup or credential deadline
`orders.order.delivered`	Buyer and seller	dispute window (physical)
`orders.escrow.released`, `.refunded`, `.partially_refunded`, `.frozen`	Buyer and seller	cancel all for the order
`payments.payout.*`, `payments.refund.*`	The payee	—
`listings.listing.published`, `.removed`; `game_accounts.ownership.*`	Seller (FR-NTF-03)	—
`auctions.bid.outbid`	Previous leader; app history only (live message is Unit 3's)	—
`auctions.auction.awarded`, `.unsold`, `.offer.*`	Winner, seller, bidders	payment window (second chance)
`delivery.meetup.*`, `.handover.*`, `.shipment.status_changed`	The other party, or both (FR-NTF-03)	cancel meetup or pickup on completion

Events	Notice to, channels	Reminders scheduled or cancelled
`game_accounts.credentials.*`, `.rekey.*`, `.takeover.completed`	Seller or buyer	credential deadline; takeover window (re-armed by a re-key); protection period
Unit 7 dispute and report events	Both parties; the reporter (FR-TNS-09)	—

Ending-soon notices (FR-NTF-02) are not event-driven: the reminder job asks Unit 3 for auctions ending within CP-38 (5 minutes) and notifies their watchers and bidders once.

3.5 Ports required

Port	Provider and use
`IdentityQuery` (Unit 1)	Display names and badges; new `getContact(userId)` for the e-mail address and number at send time, so contact details are never stored with a notification (I-36).
`FileVault` (Unit 1)	Image re-encoding and metadata removal, reused for chat images on their own volume.
`ListingsQueries`, `AuctionQuery` (Unit 3)	The listing and seller of a new thread; auctions ending soon and their bidders (I-38).
Deadline queries (Units 2, 4, 5)	Before sending a reminder: does the deadline still apply? (Section 6.6).
`ReportCommands` (Unit 7)	File a report from chat into the report queue (FR-CHT-06, FR-TNS-01).
`RateLimiter`, `AuditService`, `ConfigPort`, `Clock`	Chat limit CP-35; audit of staff reads; parameters; time.

4 Data Design

The schema is in `docs/design/sql/06_chat_notifications.sql` (Appendix B), loaded after Units 1 to 5. The unit owns two schemas, `chat` and `notify`. Its foreign keys point to users and staff (Unit 1) and listings (Unit 3). Orders and cases of later units are referenced by plain ids (DC-5).

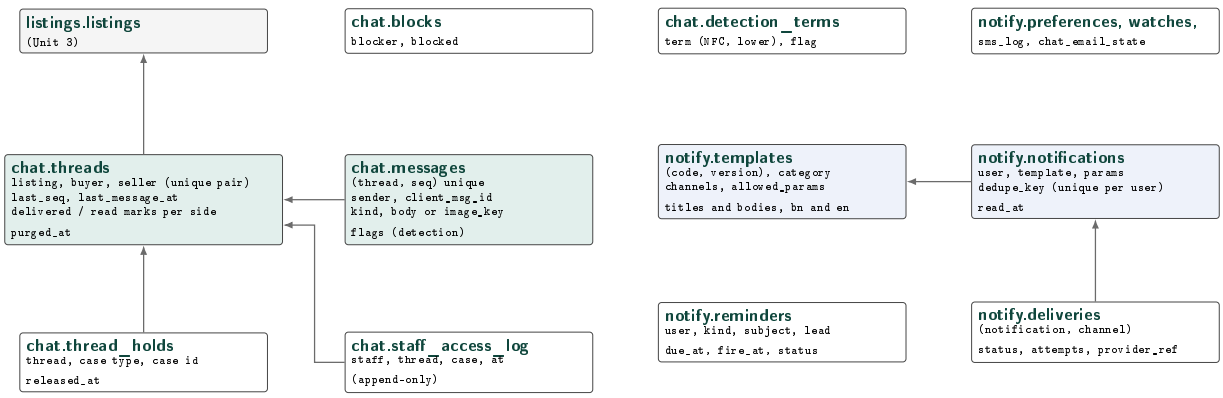


Figure 2: The `chat` and `notify` schemas. User references are omitted.

4.1 Tables and rules the database enforces

Table	Rules enforced by the database
`chat.threads`	One per listing and buyer; buyer is not the seller; parties fixed. Sequence and delivered and read marks only move forward; read ≤ delivered ≤ last message. Nothing can be added once purged.
`chat.messages`	Unique sequence number per thread, and unique client id per sender, so a resend is recognised. Text is plain, without control characters, and not empty; an image needs a stored image. Messages can never be edited or deleted; at the end of retention only their content is removed.
`chat.blocks`	Not oneself; while a block exists in either direction, no message and no new thread between the two.
`chat.thread_holds`, `chat.staff_access_log`	Staff read a thread only through a live hold of the same case; each read is logged; the log is append-only.
`chat.detection_terms`	Stored lower case and NFC-normalised.
`notify.templates`	SMS only for security notices; security notices always by SMS and e-mail; one active version per code.
`notify.notifications`	One per user and dedupe key. Parameters only those the template declares, as plain values, and none that contains a phone number, e-mail address, NID-like number or six-digit code (trigger).
`notify.deliveries`	One per notification and channel; a sent delivery records when.
`notify.reminders`	One per user, kind, subject and lead time; fire time = deadline − lead.

5 Chat

5.1 Threads

A thread belongs to one listing and one prospective buyer, with the listing's seller (FR-CHT-01). It is opened only from a listing page, by a signed-in user who is not the seller; `chat.open_thread` returns the existing thread if there is one, so ten simultaneous requests produced one thread. The same thread continues after a purchase, so the conversation before and after the sale is in one place for both parties and, if needed, for a dispute.

5.2 Sending and delivery

1. The client gives every message a random id before sending. If the network fails and the client sends again, the database recognises the id and returns the original message (tested: ten simultaneous resends stored one message).
2. `chat.post` locks the thread, checks that the sender is a participant and that no block exists, gives the message the next sequence number, runs the detector, and stores the message with its flags. Inside the database this takes about 0.33 ms (1,000 messages; three runs 0.32–0.36 ms), and twenty simultaneous posts got sequence numbers 1 to 20 with no gap or repeat.
3. After commit, the gateway emits `chat.message` to `user:{buyer}` and `user:{seller}` through the Redis adapter, so it reaches the recipient on whichever backend instance holds the connection (NFR-PERF-04, NFR-SCL-01).
4. The recipient's client acknowledges "delivered up to n" on arrival and "read up to n" when shown. The marks only move forward, so acknowledgements arriving out of order cannot move them back (tested with twenty simultaneous acknowledgements).

Sequence numbers make reconnection simple (FR-CHT-02, IF-COM-02). A client that was offline asks for each thread's messages after the last sequence number it holds, in pages of DV-44 (200).

A client that sees a gap in the live stream fetches the missing range. Delivered and read indicators (FR-CHT-05) are derived from the marks: a message is delivered if its number is at or below the recipient's delivered mark.

Chat is limited to 30 messages per minute per user (CP-35, SEC-ABU-01) by Unit 1's shared rate limiter.

5.3 Text and images

Messages are plain text of at most 2,000 characters (DV-40). The client renders them as text only, never as HTML, and web addresses are shown as non-clickable text with a warning (FR-CHT-09). Images go through the same checks as KYC images in Unit 1's `FileVault`: type identified from content, size limit CP-33, decoded, rotated upright, all metadata including location removed, re-encoded (FR-CHT-03, SEC-INP-04, SEC-DAT-08). They are stored on a private `chat_media` volume and shown only to the two participants through signed links valid for CP-31.

5.4 Detecting off-platform payment and contact

Every message passes through `chat.detect`, which returns flags. A flagged message is still delivered (FR-CHT-07), but both participants see a warning on it: *"Payments made outside TrustBazaar are not protected. Pay only through the app."* The flags and their tests are:

Flag	Detected as	Tested with
`phone`	An 11-digit Bangladeshi mobile number (`01[3-9]`...), with or without 88 or 880, also written in Bangla digits or split by spaces, dots or dashes. bKash, Nagad and Rocket wallets are mobile numbers.	`01711-234567` in Bangla digits; `+880 1811 234 567`
`account_number`	Any other run of 10 to 20 digits, after joining separated groups.	`A/C 1234 5678 9012 34`; not a price or a year
`link`	A web address or a short-link or messaging domain (`wa.me`, `t.me`).	`www.my-shop.com`
`email`	An e-mail address.	`a.b@example.com`
`payment_phrase`	A detection term asking for advance or direct payment, in Bangla or English.	*agrim*, *age taka*, "pay first"
`off_platform_app`	A term naming another messaging app.	WhatsApp, imo, Telegram
`credentials`	A term for login details (FR-GAM-20): the seller is directed to the sealed handover of Unit 5.	"login id", "pass:", *password* in Bangla

Before matching, the text is NFC-normalised, lower-cased, and Bangla digits are read as digits. English terms must stand as whole words, so "kimono" does not match "imo" and "advanced" does not match "advance" (tested). Bangla terms match as substrings, because the regular-expression engine does not treat Bangla vowel signs as word characters.

The terms are data, maintained by operators, and seeded with 31 Bangla and English terms (DV-45). False positives are accepted by design: a buyer who writes "I will pay with bKash in the app" sees the warning too. Because a flag only adds a warning and never blocks, a false positive costs a sentence of reading, while a missed scam costs money. Flagged messages also raise `chat.message.flagged`, which Unit 7 can use as a Trust & Safety signal; the event carries the flags, not the text.

5.5 Blocking

A user can block another (FR-CHT-08). While a block exists in either direction:

- no message can be sent in either direction, and no new thread opened between them (database rule; the blocker unblocks to talk again);
- the blocked user cannot bid on the blocker's auctions or buy from the blocker. Units 2 and 3 call `BlockPort.assertNotBlocked` before checkout and before a bid (I-37, I-38).

Orders already paid continue (FR-CHT-08). They need no chat: the meetup proposals and handover code (Unit 4), the sealed handover (Unit 5) and disputes (Unit 7) all work in the order view.

The bid check is made by the service before it calls `auctions.place_bid`, not inside that function, because the bid engine must not read another unit's tables (DC-1). A bid placed in the same instant as the block can therefore still be accepted. This is acceptable: a block protects against unwanted contact, not against a money invariant.

5.6 Reports, holds and staff access

A participant can report a thread or a message (FR-CHT-06). The report goes to Unit 7's queue through `ReportCommands`, and Unit 7 places a *hold* on the thread for that case. A dispute on an order places a hold on its thread in the same way. A hold has two effects:

- the thread is kept beyond the retention period while the case is open (FR-CHT-04);
- it is the only ground on which an operator may read the thread. `chat.staff_read` requires a live hold of the very case the operator is working on, and logs every read (FR-CHT-10, SEC-LOG-01). An operator with a different case, or after the hold is released, is refused (tested).

5.7 Retention and closed accounts

A thread is kept for 1 year after its last message (SRS retention schedule). The retention job then removes the content of every message (text and image) and returns the image keys so the files can be deleted, unless a hold is live. The thread row and message metadata stay, so sequence numbers and the staff access log still make sense. When an account closes, the other party sees the closed user's name as "Closed account". The content stays until the thread's retention ends, because it is also the other party's record (FR-PRV-04 permits data kept under the retention schedule).

6 Notifications

6.1 From event to notice

1. An event arrives from another unit's outbox (at least once, DC-6).
2. Its consumer decides the recipients and the template (Section 3.4), and calls `notify.create` with a dedupe key made of the event id, the recipient and the template. A second delivery of the same event creates nothing; ten simultaneous deliveries created one notification and one e-mail delivery (tested).
3. `notify.create` stores the notification for the in-app history and adds one delivery row per further channel, according to the template and the user's preferences.
4. After commit, the gateway pushes `notification.new` to the user's room. The delivery worker sends the e-mail and SMS rows.

Notifications store the template and its parameters, not rendered text. The text is rendered in the recipient's language, Bangla by default, when shown or sent. A corrected template version therefore applies to the history too, and nothing personal is stored twice.

6.2 Channels

Category	In app	E-mail	SMS
Security (FR-NTF-06)	always	always	always; cannot be switched off
Transaction, account (FR-NTF-03)	always	unless turned off	never
Chat	always	coalesced (DV-41)	never
Auction, reminder	always	unless turned off	never
OTP (Unit 1)	—	—	sent synchronously; no notification row; content never stored

The database enforces the SMS rule (FR-NTF-05): a template that is not a security notice cannot list SMS, and a security template must list both SMS and e-mail. A user who turns transactional e-mails off still gets security e-mails and SMS (tested).

- **E-mail** (IF-SW-06) goes through an adapter for SMTP or a provider API. In the academic deliverable, and in every sandbox build, the adapter writes each message to a development mailbox inside the backend, viewable on a staff-only page and labelled as simulated. This keeps within the four containers of NFR-MNT-01. E-mails are plain text with a simple HTML part, contain links only to TrustBazaar's own domain, and carry no tracking pixel. Production needs SPF, DKIM and DMARC on the sending domain.
- **SMS** (IF-SW-05) goes through an adapter for a gateway with a test mode, or a logged mock labelled as simulated (AS-4). The SMS log records an HMAC of the number, the purpose (OTP or security notice), the status and the provider reference, never the text. This keeps OTPs out of the database entirely (SEC-AUTH-09, SEC-DAT-07).
- **Retries.** Failed deliveries are retried with exponential backoff and jitter, from 1 minute to 30 minutes, six attempts (DV-43). A security notice that finally fails is shown to an operator. Provider receipts and bounces arrive on signed webhook endpoints (IF-COM-03).
- **Addresses at send time.** The e-mail address and number are fetched from Unit 1 when the delivery is sent, so a change of contact takes effect at once. A contact-change notice goes to the old contact, as Unit 1 requires.

6.3 Nothing secret in a notice

FR-NTF-08 forbids passwords, OTPs, game-account credentials, full NID numbers and other users' contact details in any notification. The design has three layers:

1. **Declared parameters.** Each template lists the parameters it may carry, and the database refuses any other, and any nested value (tested). No template declares a parameter for a password, credential, NID or contact.
2. **A content check.** The database also refuses a parameter that contains a phone number (including Bangla digits and separators), an e-mail address, a 10-, 13- or 17-digit number (the NID formats) or a six-digit code (tested). Amounts are passed as numbers, so they are not affected.
3. **Masking user text.** Listing titles and display names are written by users and may contain a phone number. The consumers pass them through `notify.mask`, which replaces such content with "[hidden]".

Chat notices never quote the message: they say "New message from Rahim about Used phone", because the message itself may hold a phone number and a lock-screen preview is visible to anyone nearby.

6.4 Chat e-mails

A new chat message always produces an in-app notice. It produces an e-mail only if the recipient is not connected at that moment and has had no chat e-mail for that thread in the last 15 minutes (DV-41). This keeps a lively conversation from producing dozens of e-mails.

6.5 Auctions

The live outbid message is Unit 3's (FR-NTF-01). This unit adds it to the history. For FR-NTF-02, users can watch an auction. Every minute, the reminder job asks Unit 3 for auctions ending within CP-38 (5 minutes) and notifies their watchers and bidders, once per auction and user (dedupe key).

6.6 Deadline reminders

FR-NTF-07 asks for reminders 24 hours and 2 hours before each deadline (CP-38): the payment window, meetup handover, courier pickup, credential delivery, takeover window, dispute window and protection period.

- **Scheduling.** The consumers schedule two reminders per deadline from the event that sets it. A deadline that moves reschedules its reminders instead of adding new ones. A deadline that moves later re-arms a reminder already sent, for example when a re-key restarts the takeover window (tested).
- **Cancelling.** The event that completes the step cancels the remaining reminders for it, for example `orders.order.paid` for the payment window.
- **Sending.** The job claims due reminders with `FOR UPDATE SKIP LOCKED`. Five job instances claiming 50 due reminders at the same moment sent each exactly once (tested). A reminder whose deadline has already passed is cancelled, never sent. Before sending, the job asks the owning unit whether the deadline still applies, so a reminder that missed its cancellation event is dropped rather than sent.

7 Scheduled Jobs

Job	Every	Action
Delivery worker	continuous	Send pending e-mail and SMS deliveries; retry failed ones (DV-43).
Reminders and ending-soon	1 min	Claim due reminders, verify, send; ending-soon notices (FR-NTF-02, FR-NTF-07).
Chat retention	daily	`chat.purge_due(1 year)`; delete the returned image files.
Notification retention	daily	Delete in-app history older than 1 year, and delivery and SMS records older than 90 days (DV-42).

8 Performance and Scaling

- **Chat latency** (NFR-PERF-04, 1 s p95). The path is: authenticated socket, rate-limit counter in Redis, one database call (about 0.33 ms inside the database), commit, and one emit through the Redis adapter to the two user rooms. No step waits on e-mail or SMS, which are sent afterwards by the worker.
- **Several instances** (NFR-SCL-01). No instance holds chat state: sequence numbers come from the database, rooms are joined per connection, and messages between instances pass through Redis. Jobs use `SKIP LOCKED`.

- **Restart.** Messages are committed before they are emitted, so a restart loses only connections; clients reconnect and fetch after their last sequence number (IF-COM-02).
- **Load test** (to run with the built application): 500 signed-in users with chat open, 50 messages per second across 100 threads, measuring sender-to-recipient time at the clients; and a burst of 5,000 notifications from one event storm, measuring the time to the last in-app push.

9 Security and Privacy

- **Who can read a thread.** Only its two participants: the database checks the sender of every message and every acknowledgement, and the socket joins only the user's own room (SEC-SES-07). Staff read only through a held case, and every read is logged (FR-CHT-10).
- **Content safety.** Plain text only, rendered as text; links shown inert; images re-encoded without metadata and served through signed links to participants only (SEC-INP-04, SEC-DAT-08).
- **Fraud signals.** Off-platform and credential warnings on the message itself, for both parties; flags, not text, go to Trust & Safety.
- **Notices.** No secret in a notice (three layers), no message text in chat notices, no OTP or SMS text stored, contact details fetched at send time.
- **Abuse limits.** Chat 30 messages per minute (CP-35); reports 20 per day (CP-35); OTP send limits stay in Unit 1 (CP-05). Retries and coalescing keep a fault from flooding a user's inbox.
- **Integrity.** Messages cannot be edited or deleted by anyone; only retention removes their content. The staff access log is append-only, and the application role cannot change it or delete messages (tested).

10 Parameters

SRS parameters used: CP-05 (OTP sends, applied by Unit 1); CP-31 (signed links); CP-33 (image size); CP-35 (chat and report limits); CP-38 (lead times); CP-41 (closure).

Design values introduced (continuing DV-1 to DV-39):

ID	Value	Default	Reason
DV-40	Message size	2,000 characters; image caption 300; one image per message	Chat, not documents.
DV-41	Chat e-mail coalescing	at most one per thread per 15 min, only when offline	No inbox flooding.
DV-42	Notification retention	in-app history 1 year; delivery and SMS records 90 days	Not in the SRS schedule (I-42).
DV-43	E-mail and SMS retries	1 min doubling to 30 min; 6 attempts	Then failed; security failures to an operator.
DV-44	Sync page size	200 messages; 50 notifications	Fast reconnect on mobile.
DV-45	Detection terms	31 seeded terms, Bangla and English	Maintained by operators.
DV-46	Ending-soon recipients	watchers and bidders, once per auction	FR-NTF-02 names both.

11 Testing

Level	Cases
Database (done)	69 checks in `06_chat_notifications_tests.sql`, each rejection required to fail on its intended rule: threads and their uniqueness; participants only; sequence numbers; resend recognition; plain text, empty and image messages; delivered and read marks; no editing or deletion; detection of phones in Bangla and English digits, account numbers, links, e-mail, Bangla and English phrases, other apps and login details, with whole-word matching; blocking both ways and unblocking; staff reads only through a held case; retention with and without holds; the channel rules of templates; dedupe; parameter whitelist and content checks; masking; preferences and the security exception; history; reminders scheduled, moved, re-armed, cancelled and claimed once; chat e-mail coalescing; watches; grants. The Unit 1 to 5 scripts still pass with this schema loaded.
Concurrency (done)	Six races, three runs each on fresh databases, same results every time: 20 simultaneous posts (sequence 1 to 20, no gap); 10 resends (one message); 10 thread openings (one thread); one event consumed 10 times (one notice); 5 job instances and 50 reminders (each sent once); 20 out-of-order acknowledgements (the marks end at the highest). Timing 0.32–0.36 ms per message in the database.
Unit	The detector's TypeScript tests reuse the SQL test sentences; template rendering in both languages; masking of titles; the coalescing rule with and without a connected socket.
Integration	Message to an online recipient on another instance through Redis; reconnect and resync after a gap; block stops chat and, through `BlockPort`, checkout and bidding; a dispute's hold allows exactly that operator's read; OTP SMS leaves no content in any table.
Load	The load test of the performance section.

12 Notes for Integration

I-36 Unit 1. `NotificationPort` is implemented here; OTP rate limits stay in Unit 1. New `IdentityQuery.getContact(userId)` returns the e-mail address and number at send time. On `identity.account.closed`, chat shows "Closed account", and the user's preferences and watches are deleted.

I-37 Unit 2. Call `BlockPort.assertNotBlocked` when creating an order and when accepting a second-chance offer (FR-CHT-08). Order and escrow events must carry the deadlines they set (`payment_due_at`, `fulfilment_due_at`, `dispute_window_ends_at`, `auto_release_at`) and the listing title. Provide a deadline query for the reminder job.

I-38 Unit 3. Call `BlockPort.assertNotBlocked` in `BidService` before `auctions.place_bid`. New `AuctionQuery.endingBetween(from, to)` and `bidders(auctionId)` for FR-NTF-02.

I-39 Units 4 and 5. Their events already carry the deadlines (meetup, pickup, credential delivery, takeover window, re-key); this unit schedules and cancels reminders from them. Notices about the handover code and sealed credentials say only that something happened, never the code, key or fingerprint.

I-40 Unit 7. Use `ChatCommands.hold` and `release` for every dispute and report that concerns a thread, and `chat.staff_read` in the case file (FR-DSP-05). Implement `ReportCommands` for FR-CHT-06. Notify parties and reporters through `Notifier` (FR-TNS-09). Use `chat.message.flagged` as a signal.

I-41 Unit 8. Render chat as text only; give every message a client id; resync after reconnects and gaps; show the warnings on flagged messages to both parties; offer block and report on every thread; provide the notification centre and preferences, in Bangla and English.

I-42 SRS clarifications proposed:

- FR-CHT-08: the block stops messages in both directions, while paid orders continue

through the order view;
- FR-NTF-05: users may turn transactional e-mails off; security notices cannot be turned off;
- retention schedule: add notifications and delivery records (DV-42);
- FR-CHT-07: detection is advisory and false positives are expected;
- add DV-40 to DV-46 to the SRS parameter table.

A Traceability

Requirement	Design element	Section
FR-CHT-01	`chat.threads`, one per listing and buyer; `open_thread`	5
FR-CHT-02	`chat.post`; user rooms through Redis; resync after a sequence number	5
FR-CHT-03	Image pipeline; private volume; signed links	5.3
FR-CHT-04	Messages never edited or deleted; retention with holds	5.6
FR-CHT-05	Delivered and read marks per participant	5
FR-CHT-06	Report through `ReportCommands`; hold	5.6
FR-CHT-07	`chat.detect`; warnings to both; delivered anyway	5.4
FR-CHT-08	`chat.blocks`; `BlockPort` for Units 2 and 3	5.5
FR-CHT-09	Plain text; inert links	5.3
FR-CHT-10	`chat.staff_read` only through a held case; access log	5.6
FR-NTF-01	Outbid history (live message in Unit 3)	6
FR-NTF-02	Watches; ending-soon notices to watchers and bidders	6
FR-NTF-03	Event-to-notice map	3.4
FR-NTF-04	`notify.notifications`; history API	6
FR-NTF-05	Channel rules (database); preferences	6.2
FR-NTF-06	Security category: SMS and e-mail, not switchable	6.2
FR-NTF-07	`notify.reminders`; schedule, move, cancel, claim, verify	6.6
FR-NTF-08	Declared parameters; content check; masking; no chat text	6
FR-GAM-20	`credentials` flag; pointer to the sealed handover	5.4
FR-TNS-01, 09	Reports from chat; reporter notified through `Notifier`	5.6, 12
FR-PRV-04	Closed accounts in chat; retention	5.6
SEC-SES-07	User rooms only; participant checks in the database	3.1
SEC-ABU-01	Chat and report limits	5
SEC-DAT-07	No OTP or SMS text stored; nothing secret in notices	6.2
SEC-DAT-08	Image metadata removed	5.3
SEC-LOG-01	Staff access log	5.6
IF-SW-05, 06	SMS and e-mail adapters; dev mailbox; mock SMS	6.2
IF-COM-02	Resync by sequence number	5
IF-COM-03	Signed receipt webhooks	6.2
NFR-PERF-04	0.33 ms in the database; emit after commit; load test	11

B Database Schema and Tests

B.1 Test results

Loaded after the Unit 1 to Unit 5 schemas in PostgreSQL 16, using PostgreSQL 15 features only. Every check passed. The Bangla test sentences are in the test script; the output below shows only the labels.

```
PASS  FR-CHT-01 a seller cannot open a thread with themselves [thread_parties]
PASS  FR-CHT-01 one thread per listing and buyer: opening again returns the same thread
PASS  thread parties are fixed [parties are fixed]
PASS  SEC-SES-07 only the two participants can write [only the two participants]
```

```
PASS  messages get consecutive sequence numbers in the thread
PASS  the sender's own messages count as delivered and read for the sender
PASS  a resend with the same client id returns the original, not a copy
PASS  FR-CHT-09 plain text only: control characters are refused [message_plain_text]
PASS  an empty message is refused [message_content]
PASS  FR-CHT-03 an image message needs a stored image [message_content]
PASS  FR-CHT-03 an image message with a caption is stored
PASS  FR-CHT-05 the recipient's delivered and read marks are recorded
PASS  marks never move backwards
PASS  a mark beyond the last message is clamped, and read implies delivered
PASS  only a participant can acknowledge [only a participant]
PASS  watermarks cannot be moved back directly [only move forward]
PASS  FR-CHT-04 messages cannot be edited [cannot be edited]
PASS  FR-CHT-04 messages cannot be deleted [append-only]
PASS  FR-CHT-07 a Bangla-digit mobile number split by a dash is a phone number
PASS  FR-CHT-07 +880 with spaces is a phone number too
PASS  FR-CHT-07 a long digit run is an account number; a price and a year are not
PASS  FR-CHT-07 Bangla advance-payment phrases are detected
PASS  FR-CHT-07 English payment phrases and other apps are detected
PASS  Latin terms must be whole words: kimono, advanced and passing are not flagged
PASS  FR-CHT-09 links and e-mail addresses are flagged
PASS  FR-GAM-20 login details in chat are flagged (Bangla and English)
PASS  an ordinary message has no flags
PASS  FR-CHT-07 a flagged message is still delivered, with its flags, and raises an event
PASS  detection terms are stored normalised (lower case) [term_normalised]
PASS  a user cannot block themselves [block_not_self]
PASS  FR-CHT-08 a blocked user cannot send messages to the blocker [blocked]
PASS  FR-CHT-08 nor can the blocker write while the block stands [blocked]
PASS  FR-CHT-08 a blocked user cannot open a new thread with the blocker [blocked]
PASS  after unblocking, messages flow again
PASS  FR-CHT-10 staff cannot read a thread without a case that concerns it [only through an open case]
PASS  FR-CHT-10 with the dispute's hold, the thread is readable and the read is logged
PASS  ... one access-log row
PASS  a different case gives no access [only through an open case]
PASS  SEC-LOG-01 the staff access log is append-only [append-only]
PASS  retention: a silent thread past one year is purged, and its image files are returned for deletion
PASS  FR-CHT-04 a thread held by an open dispute is kept beyond retention
PASS  nothing can be posted into a purged thread [purged]
PASS  FR-NTF-05 SMS only for security notices [sms_only_for_security]
PASS  FR-NTF-06 security notices go by SMS and e-mail [security_by_sms_and_email]
PASS  FR-NTF-03 an order notice is created with an e-mail delivery
PASS  DC-6 the same event delivered twice creates one notification
PASS  FR-NTF-08 only the template's declared parameters are accepted [not allowed in template]
PASS  parameters must be plain values [plain value]
PASS  FR-NTF-08 no phone number, even inside a seller's title [contact details or a code]
PASS  FR-NTF-08 no e-mail address [contact details or a code]
PASS  FR-NTF-08 no NID-like number [contact details or a code]
PASS  FR-NTF-08 no six-digit code [contact details or a code]
PASS  a masked title passes
PASS  FR-NTF-05 a user who turned off e-mails gets transactional notices in the app only
PASS  FR-NTF-06 security notices still go by SMS and e-mail; they cannot be switched off
PASS  a sent delivery records when it was sent [delivery_sent_fields]
PASS  FR-NTF-04 the history lists the user's notifications, newest first; unread ones are counted
PASS  CP-38 two reminders, 24 h and 2 h before the deadline
PASS  a moved deadline moves its reminder (no duplicate)
PASS  the job sends only reminders that are due and whose deadline is still ahead
PASS  a claimed reminder is not sent twice
PASS  FR-GAM-12 a restarted takeover window re-arms its sent reminder
PASS  the step happening cancels the outstanding reminders
PASS  DV-41 the first chat e-mail for a thread is allowed
PASS  DV-41 a second within the window is not
PASS  FR-NTF-02 a listing is watched once per user [watches_pkey]
PASS  DC-6 outbox events are named chat.<entity>.<past> [outbox_event_type_check]
PASS  application role cannot rewrite the staff access log [permission denied]
PASS  application role cannot delete chat messages [permission denied]

69 checks passed (06_chat_notifications_tests.sql, PostgreSQL 16.15 with the Unit 1-5 schemas loaded).
The Unit 1-5 test scripts still pass with Unit 6 loaded (46, 45, 67, 93 and 70 checks).

Concurrency (06_chat_notifications_concurrency.sh). Three runs on fresh databases gave the
same results; the timing is from run 1 (runs 2 and 3: 0.323 and 0.348 ms).
  Posting race: buyer and seller post 20 messages into one thread at once
    database: messages=20, seq 1..20, distinct seq=20
  Resend race: one message resent 10 times at once (same client id)
    results: stored 1, recognised as duplicates 9
```

```
  Thread race: 10 requests open the same thread at once
    results: distinct thread ids returned 1
    database: threads=1
  Notification race: the same event consumed 10 times at once
    results: created 1, duplicates 9
    database: notifications=1, deliveries=1
  Reminder race: 5 job instances claim 50 due reminders at once (20 each at most)
    results: claimed 50 in total
    database: sent=50, still scheduled=0
  Acknowledgement race: 20 acknowledgements (delivered N, read N-1) arriving at once
    database: delivered=20, read=19, last=20
  Timing: chat.post inside the database, 1000 messages in one session
    0.355 ms per message (excluding network and commit)
```

B.2 Schema

In this printout, Bangla characters in the detection terms and the digit table are shown as their Unicode code points (<U+0985> and so on); the file itself contains the Bangla text.

```
-- ====================================================================
-- TrustBazaar - Unit 6: Chat & Notifications - PostgreSQL 15 schema
-- Design document: docs/design/06_chat_notifications.pdf
-- Requirements: SRS v2.0 FR-CHT-01..10, FR-NTF-01..08, FR-GAM-20, FR-PRV-04 (chat side);
--   IF-SW-05, IF-SW-06, IF-COM-02; SEC-SES-07 (chat rooms), SEC-ABU-01 (chat limit),
--   SEC-DAT-07/08, SEC-LOG-01 (staff reads); NFR-PERF-04.
-- Depends on: 01_identity_access.sql (identity.users, identity.staff_accounts,
--   identity.forbid_change), 03_listings_auctions.sql (listings.listings).
-- Conventions DC-1..DC-14. No function reads another unit's tables (DC-1).
-- ====================================================================

CREATE SCHEMA IF NOT EXISTS chat;
CREATE SCHEMA IF NOT EXISTS notify;

-- ====================================================================
-- CHAT (FR-CHT)
-- ====================================================================
CREATE TYPE chat.message_kind AS ENUM ('text', 'image');
CREATE TYPE chat.case_type    AS ENUM ('dispute', 'report');

-- one thread per listing and prospective buyer, with the seller (FR-CHT-01)
CREATE TABLE chat.threads (
  id                   uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  listing_id           uuid NOT NULL REFERENCES listings.listings(id),
  buyer_id             uuid NOT NULL REFERENCES identity.users(id),
  seller_id            uuid NOT NULL REFERENCES identity.users(id),
  created_at           timestamptz NOT NULL DEFAULT now(),
  last_seq             bigint NOT NULL DEFAULT 0 CHECK (last_seq >= 0),
  last_message_at      timestamptz,
  -- per-participant watermarks: every message with seq <= the value is delivered / read (FR-CHT-05)
  buyer_delivered_seq  bigint NOT NULL DEFAULT 0,
  buyer_read_seq       bigint NOT NULL DEFAULT 0,
  seller_delivered_seq bigint NOT NULL DEFAULT 0,
  seller_read_seq      bigint NOT NULL DEFAULT 0,
  purged_at            timestamptz,                    -- content removed at the end of retention
  CONSTRAINT thread_parties CHECK (buyer_id <> seller_id),
  CONSTRAINT thread_one_per_buyer UNIQUE (listing_id, buyer_id),
  CONSTRAINT watermarks_order CHECK (0 <= buyer_read_seq AND buyer_read_seq <= buyer_delivered_seq AND
    buyer_delivered_seq <= last_seq
                               AND 0 <= seller_read_seq AND seller_read_seq <= seller_delivered_seq AND
    seller_delivered_seq <= last_seq)
);
CREATE INDEX threads_buyer  ON chat.threads (buyer_id, last_message_at DESC);
CREATE INDEX threads_seller ON chat.threads (seller_id, last_message_at DESC);
CREATE INDEX threads_retention ON chat.threads (last_message_at) WHERE purged_at IS NULL;

CREATE FUNCTION chat.thread_guard() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF (NEW.listing_id, NEW.buyer_id, NEW.seller_id, NEW.created_at) IS DISTINCT FROM (OLD.listing_id, OLD.
    buyer_id, OLD.seller_id, OLD.created_at) THEN
    RAISE EXCEPTION 'thread␣parties␣are␣fixed' USING ERRCODE = 'check_violation';
  END IF;
  IF NEW.last_seq < OLD.last_seq OR NEW.buyer_delivered_seq < OLD.buyer_delivered_seq OR NEW.buyer_read_seq <
```

```
     OLD.buyer_read_seq
     OR NEW.seller_delivered_seq < OLD.seller_delivered_seq OR NEW.seller_read_seq < OLD.seller_read_seq THEN
    RAISE EXCEPTION 'sequence␣and␣delivery␣watermarks␣only␣move␣forward' USING ERRCODE = 'check_violation';
  END IF;
  IF OLD.purged_at IS NOT NULL AND NEW.last_seq <> OLD.last_seq THEN
    RAISE EXCEPTION 'thread␣has␣been␣purged' USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER thread_guard BEFORE UPDATE ON chat.threads FOR EACH ROW EXECUTE FUNCTION chat.thread_guard();

CREATE TABLE chat.messages (
  id             bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  thread_id      uuid NOT NULL REFERENCES chat.threads(id),
  seq            bigint NOT NULL CHECK (seq >= 1),
  sender_id      uuid NOT NULL REFERENCES identity.users(id),
  client_msg_id  uuid NOT NULL,                        -- generated by the client; makes resending safe
  kind           chat.message_kind NOT NULL,
  body           varchar(2000),                        -- plain text only (FR-CHT-09)
  image_key      varchar(80),                          -- private chat_media volume; metadata stripped (FR-
    CHT-03)
  flags          text[] NOT NULL DEFAULT '{}',         -- warnings computed by chat.detect (FR-CHT-07, FR-GAM
    -20)
  created_at     timestamptz NOT NULL DEFAULT now(),
  purged_at      timestamptz,
  CONSTRAINT message_seq_unique UNIQUE (thread_id, seq),
  CONSTRAINT message_client_unique UNIQUE (thread_id, sender_id, client_msg_id),
  CONSTRAINT message_content CHECK (purged_at IS NOT NULL OR
         (kind = 'text'  AND body IS NOT NULL AND image_key IS NULL AND length(btrim(body)) >= 1) OR
         (kind = 'image' AND image_key IS NOT NULL AND (body IS NULL OR length(body) <= 300))),
  CONSTRAINT message_plain_text CHECK (body IS NULL OR body !~ '[\x01-\x08\x0B\x0C\x0E-\x1F\x7F]'),
  CONSTRAINT message_purged_empty CHECK (purged_at IS NULL OR (body IS NULL AND image_key IS NULL))
);
CREATE INDEX messages_thread ON chat.messages (thread_id, seq);

-- messages are never edited; at the end of retention only their content is removed
CREATE FUNCTION chat.message_guard() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.purged_at IS NULL OR OLD.purged_at IS NOT NULL
     OR (NEW.thread_id, NEW.seq, NEW.sender_id, NEW.client_msg_id, NEW.kind, NEW.flags, NEW.created_at)
        IS DISTINCT FROM (OLD.thread_id, OLD.seq, OLD.sender_id, OLD.client_msg_id, OLD.kind, OLD.flags, OLD.
     created_at) THEN
    RAISE EXCEPTION 'chat␣messages␣cannot␣be␣edited' USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER message_guard BEFORE UPDATE ON chat.messages FOR EACH ROW EXECUTE FUNCTION chat.message_guard
     ();
CREATE TRIGGER message_no_delete BEFORE DELETE ON chat.messages FOR EACH ROW EXECUTE FUNCTION identity.
     forbid_change();

-- FR-CHT-08: a block stops messages in both directions until it is lifted
CREATE TABLE chat.blocks (
  blocker_id  uuid NOT NULL REFERENCES identity.users(id),
  blocked_id  uuid NOT NULL REFERENCES identity.users(id),
  created_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (blocker_id, blocked_id),
  CONSTRAINT block_not_self CHECK (blocker_id <> blocked_id)
);
CREATE INDEX blocks_blocked ON chat.blocks (blocked_id);

-- holds: a dispute or report that concerns a thread keeps it beyond retention and is the only
-- ground on which staff may read it (FR-CHT-04, FR-CHT-10)
CREATE TABLE chat.thread_holds (
  thread_id    uuid NOT NULL REFERENCES chat.threads(id),
  case_type    chat.case_type NOT NULL,
  case_id      uuid NOT NULL,                          -- Unit 7 (no FK, DC-5)
  placed_at    timestamptz NOT NULL DEFAULT now(),
  released_at  timestamptz,
  PRIMARY KEY (thread_id, case_type, case_id)
);
CREATE INDEX holds_live ON chat.thread_holds (thread_id) WHERE released_at IS NULL;

CREATE TABLE chat.staff_access_log (                                          -- SEC-LOG-01, append-only
  id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
  staff_id   uuid NOT NULL REFERENCES identity.staff_accounts(id),
  thread_id  uuid NOT NULL REFERENCES chat.threads(id),
  case_type  chat.case_type NOT NULL,
  case_id    uuid NOT NULL,
  at         timestamptz NOT NULL DEFAULT now()
);
CREATE TRIGGER staff_access_append_only BEFORE UPDATE OR DELETE ON chat.staff_access_log
  FOR EACH ROW EXECUTE FUNCTION identity.forbid_change();

-- detection terms (FR-CHT-07, FR-GAM-20), maintained by operators like Unit 3's screening terms
CREATE TABLE chat.detection_terms (
  id        integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  term      varchar(60) NOT NULL UNIQUE,      -- stored lower case, NFC
  flag      varchar(20) NOT NULL CHECK (flag IN ('payment_phrase', 'off_platform_app', 'credentials')),
  added_by  uuid REFERENCES identity.staff_accounts(id),
  added_at  timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT term_normalised CHECK (term = lower(normalize(term, NFC)))
);

CREATE TABLE chat.outbox (                                                  -- DC-6
  id              bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  event_type      varchar(80) NOT NULL CHECK (event_type ~ '^chat\.[a-z_]+\.[a-z_]+$'),
  aggregate_id    uuid NOT NULL,
  payload         jsonb NOT NULL,
  created_at      timestamptz NOT NULL DEFAULT now(),
  attempts        smallint NOT NULL DEFAULT 0,
  next_attempt_at timestamptz NOT NULL DEFAULT now(),
  dispatched_at   timestamptz
);
CREATE INDEX chat_outbox_pending ON chat.outbox (next_attempt_at) WHERE dispatched_at IS NULL;

-- ------------------------------------------------------------------ detection
-- Returns the warning flags for a text. Bangla digits are read as digits, separators between
-- digits are ignored, and Unicode is normalised (NFC). A flag never stops delivery (FR-CHT-07).
CREATE FUNCTION chat.normalise(p text) RETURNS text LANGUAGE sql IMMUTABLE AS $$
  SELECT lower(translate(normalize(p, NFC), '<U+09E6><U+09E7><U+09E8><U+09E9><U+09EA><U+09EB><U+09EC><U+09ED
    ><U+09EE><U+09EF>', '0123456789'))
$$;

CREATE FUNCTION chat.detect(p_body text) RETURNS text[] LANGUAGE plpgsql STABLE AS $$
DECLARE
  t text; digits text; run text; f text[] := '{}';
BEGIN
  IF p_body IS NULL THEN RETURN f; END IF;
  t := chat.normalise(p_body);
  -- join digit groups split by spaces, dots, dashes or a leading +, then inspect each run
  digits := regexp_replace(t, '(?<=[0-9])[␣.\-]+(?=[0-9])', '', 'g');
  FOR run IN SELECT m[1] FROM regexp_matches(digits, '([0-9]+)', 'g') AS m LOOP
    IF run ~ '^(880|88)?01[3-9][0-9]{8}$' THEN
      f := f || 'phone'::text;                          -- mobile and MFS wallet numbers
    ELSIF length(run) BETWEEN 10 AND 20 THEN
      f := f || 'account_number'::text;                 -- bank accounts, card-like numbers
    END IF;
  END LOOP;
  IF t ~ '(https?://|www\.|\m(wa\.me|t\.me|bit\.ly)\M|\m[a-z0-9-]+\.(com|net|org|bd|me|io|xyz|link|shop|store
    |app)\M)' THEN
    f := f || 'link'::text;
  END IF;
  IF t ~ '[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}' THEN
    f := f || 'email'::text;
  END IF;
  -- Latin terms must stand as words ("imo" must not match "kimono"); Bangla terms match as
  -- substrings, because Bangla vowel signs are not word characters to the regex engine
  SELECT f || coalesce(array_agg(DISTINCT d.flag), '{}') INTO f
    FROM chat.detection_terms d
   WHERE CASE WHEN d.term ~ '^[␣-~]+$'
              THEN t ~ ('(^|[^a-z0-9])' || regexp_replace(d.term, '([.^$*+?()\[\]{}|\\])', '\\\1', 'g')
                        || CASE WHEN d.term ~ '[a-z0-9]$' THEN '($|[^a-z0-9])' ELSE '' END)
              ELSE position(d.term IN t) > 0 END;
  RETURN ARRAY(SELECT DISTINCT x FROM unnest(f) x ORDER BY x);
END $$;

-- ------------------------------------------------------------------ operations
-- FR-CHT-01: open (or find) the thread; safe when two requests race
CREATE FUNCTION chat.open_thread(p_listing uuid, p_buyer uuid, p_seller uuid) RETURNS uuid LANGUAGE plpgsql
```

```
    AS $$
DECLARE tid uuid;
BEGIN
  IF EXISTS (SELECT 1 FROM chat.blocks WHERE (blocker_id, blocked_id) IN ((p_buyer, p_seller), (p_seller,
    p_buyer))) THEN
    RAISE EXCEPTION 'messaging␣is␣blocked␣between␣these␣users' USING ERRCODE = 'insufficient_privilege';
  END IF;
  INSERT INTO chat.threads (listing_id, buyer_id, seller_id) VALUES (p_listing, p_buyer, p_seller)
  ON CONFLICT ON CONSTRAINT thread_one_per_buyer DO NOTHING RETURNING id INTO tid;
  IF tid IS NULL THEN
    SELECT id INTO tid FROM chat.threads WHERE listing_id = p_listing AND buyer_id = p_buyer;
  END IF;
  RETURN tid;
END $$;

-- FR-CHT-02, FR-CHT-07: post a message. Returns its sequence number; a resend with the same
-- client id returns the original message instead of a copy.
CREATE FUNCTION chat.post(p_thread uuid, p_sender uuid, p_client_id uuid, p_kind chat.message_kind,
                          p_body text, p_image_key varchar DEFAULT NULL)
RETURNS TABLE (message_id bigint, seq bigint, flags text[], duplicate boolean) LANGUAGE plpgsql AS $$
DECLARE th chat.threads; m chat.messages; other uuid;
BEGIN
  SELECT * INTO th FROM chat.threads WHERE id = p_thread FOR UPDATE;
  IF NOT FOUND OR p_sender NOT IN (th.buyer_id, th.seller_id) THEN
    RAISE EXCEPTION 'only␣the␣two␣participants␣can␣write␣in␣a␣thread' USING ERRCODE = 'insufficient_privilege
    ';
  END IF;
  SELECT * INTO m FROM chat.messages WHERE thread_id = p_thread AND sender_id = p_sender AND client_msg_id =
    p_client_id;
  IF FOUND THEN
    RETURN QUERY SELECT m.id, m.seq, m.flags, true; RETURN;
  END IF;
  IF th.purged_at IS NOT NULL THEN RAISE EXCEPTION 'thread␣has␣been␣purged' USING ERRCODE = 'check_violation'
    ; END IF;
  other := CASE WHEN p_sender = th.buyer_id THEN th.seller_id ELSE th.buyer_id END;
  IF EXISTS (SELECT 1 FROM chat.blocks WHERE (blocker_id, blocked_id) IN ((p_sender, other), (other, p_sender
    ))) THEN
    RAISE EXCEPTION 'messaging␣is␣blocked␣between␣these␣users' USING ERRCODE = 'insufficient_privilege';
  END IF;
  INSERT INTO chat.messages (thread_id, seq, sender_id, client_msg_id, kind, body, image_key, flags)
  VALUES (p_thread, th.last_seq + 1, p_sender, p_client_id, p_kind, p_body, p_image_key, chat.detect(p_body))
  RETURNING * INTO m;
  UPDATE chat.threads SET last_seq = m.seq, last_message_at = m.created_at,
         buyer_delivered_seq  = CASE WHEN p_sender = buyer_id  THEN m.seq ELSE buyer_delivered_seq END,
         buyer_read_seq       = CASE WHEN p_sender = buyer_id  THEN m.seq ELSE buyer_read_seq END,
         seller_delivered_seq = CASE WHEN p_sender = seller_id THEN m.seq ELSE seller_delivered_seq END,
         seller_read_seq      = CASE WHEN p_sender = seller_id THEN m.seq ELSE seller_read_seq END
   WHERE id = p_thread;
  IF cardinality(m.flags) > 0 THEN
    INSERT INTO chat.outbox (event_type, aggregate_id, payload)
    VALUES ('chat.message.flagged', p_thread, jsonb_build_object('messageId', m.id, 'flags', m.flags, 'sender
    ', p_sender));
  END IF;
  RETURN QUERY SELECT m.id, m.seq, m.flags, false;
END $$;

-- FR-CHT-05: the recipient's client reports what it has received and shown; values only move forward
CREATE FUNCTION chat.mark(p_thread uuid, p_user uuid, p_delivered bigint, p_read bigint) RETURNS void
    LANGUAGE plpgsql AS $$
DECLARE th chat.threads; d bigint; r bigint;
BEGIN
  SELECT * INTO th FROM chat.threads WHERE id = p_thread FOR UPDATE;
  IF NOT FOUND OR p_user NOT IN (th.buyer_id, th.seller_id) THEN
    RAISE EXCEPTION 'only␣a␣participant␣can␣acknowledge␣messages' USING ERRCODE = 'insufficient_privilege';
  END IF;
  d := least(greatest(p_delivered, p_read), th.last_seq);
  r := least(p_read, th.last_seq);
  IF p_user = th.buyer_id THEN
    UPDATE chat.threads SET buyer_delivered_seq = greatest(buyer_delivered_seq, d),
                            buyer_read_seq = greatest(buyer_read_seq, r) WHERE id = p_thread;
  ELSE
    UPDATE chat.threads SET seller_delivered_seq = greatest(seller_delivered_seq, d),
                            seller_read_seq = greatest(seller_read_seq, r) WHERE id = p_thread;
  END IF;
END $$;
```

```
-- FR-CHT-10: an operator reads a thread only for a case that holds it; every read is logged
CREATE FUNCTION chat.staff_read(p_thread uuid, p_staff uuid, p_case_type chat.case_type, p_case uuid)
RETURNS SETOF chat.messages LANGUAGE plpgsql AS $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM chat.thread_holds WHERE thread_id = p_thread AND case_type = p_case_type
                 AND case_id = p_case AND released_at IS NULL) THEN
    RAISE EXCEPTION 'staff may read a thread only through an open case that concerns it' USING ERRCODE = '
     insufficient_privilege';
  END IF;
  INSERT INTO chat.staff_access_log (staff_id, thread_id, case_type, case_id) VALUES (p_staff, p_thread,
     p_case_type, p_case);
  RETURN QUERY SELECT * FROM chat.messages WHERE thread_id = p_thread ORDER BY seq;
END $$;

-- retention (FR-CHT-04, SRS retention schedule): content of threads silent for p_after with no
-- live hold is removed. Returns the image keys whose files the service must delete.
CREATE FUNCTION chat.purge_due(p_after interval) RETURNS SETOF varchar LANGUAGE plpgsql AS $$
DECLARE th record;
BEGIN
  FOR th IN SELECT t.id FROM chat.threads t
            WHERE t.purged_at IS NULL AND t.last_message_at < now() - p_after
              AND NOT EXISTS (SELECT 1 FROM chat.thread_holds h WHERE h.thread_id = t.id AND h.released_at
     IS NULL)
            FOR UPDATE SKIP LOCKED LOOP
    RETURN QUERY SELECT m.image_key FROM chat.messages m WHERE m.thread_id = th.id AND m.image_key IS NOT
     NULL;
    UPDATE chat.messages SET body = NULL, image_key = NULL, purged_at = now() WHERE thread_id = th.id AND
     purged_at IS NULL;
    UPDATE chat.threads SET purged_at = now() WHERE id = th.id;
  END LOOP;
END $$;

-- ================================================================
-- NOTIFICATIONS (FR-NTF)
-- ================================================================
CREATE TYPE notify.category AS ENUM ('security', 'transaction', 'chat', 'auction', 'reminder', 'account');
CREATE TYPE notify.channel  AS ENUM ('email', 'sms');                -- in-app is implicit: every
     notification
CREATE TYPE notify.delivery_status AS ENUM ('pending', 'sent', 'failed', 'suppressed');
CREATE TYPE notify.reminder_kind AS ENUM
  ('payment_window', 'meetup_handover', 'courier_pickup', 'credential_delivery', 'takeover_window',
   'dispute_window', 'protection_period');                           -- FR-NTF-07
CREATE TYPE notify.reminder_status AS ENUM ('scheduled', 'sent', 'cancelled');

-- templates in Bangla and English, each declaring the only parameters it may carry (FR-NTF-08)
CREATE TABLE notify.templates (
  code            varchar(60) NOT NULL CHECK (code ~ '^[a-z_]+\.[a-z_]+$'),
  version         smallint NOT NULL CHECK (version >= 1),
  category        notify.category NOT NULL,
  channels        notify.channel[] NOT NULL DEFAULT '{}',
  allowed_params  text[] NOT NULL DEFAULT '{}',
  title_en        varchar(120) NOT NULL,
  title_bn        varchar(160) NOT NULL,
  body_en         varchar(1000) NOT NULL,
  body_bn         varchar(1500) NOT NULL,
  active          boolean NOT NULL DEFAULT true,
  PRIMARY KEY (code, version),
  -- FR-NTF-05: SMS only for security notices (OTPs bypass this table entirely)
  CONSTRAINT sms_only_for_security CHECK (NOT ('sms' = ANY (channels)) OR category = 'security'),
  -- FR-NTF-06: security notices go by SMS and e-mail
  CONSTRAINT security_by_sms_and_email CHECK (category <> 'security' OR (channels @> ARRAY['sms', 'email']::
     notify.channel[]))
);
CREATE UNIQUE INDEX templates_one_active ON notify.templates (code) WHERE active;

CREATE TABLE notify.preferences (
  user_id             uuid PRIMARY KEY REFERENCES identity.users(id),
  language            varchar(2) NOT NULL DEFAULT 'bn' CHECK (language IN ('bn', 'en')),
  email_transactional boolean NOT NULL DEFAULT true,     -- may be turned off; security notices ignore it
  updated_at          timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE notify.notifications (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  user_id           uuid NOT NULL REFERENCES identity.users(id),
  template_code     varchar(60) NOT NULL,
  template_version  smallint NOT NULL,
  params            jsonb NOT NULL DEFAULT '{}' CHECK (jsonb_typeof(params) = 'object'),
  dedupe_key        varchar(200) NOT NULL,              -- source event id + recipient + kind: consumers are
     idempotent
  created_at        timestamptz NOT NULL DEFAULT now(),
  read_at           timestamptz,
  FOREIGN KEY (template_code, template_version) REFERENCES notify.templates(code, version),
  CONSTRAINT notification_once UNIQUE (user_id, dedupe_key)
);
CREATE INDEX notifications_user ON notify.notifications (user_id, created_at DESC);
CREATE INDEX notifications_unread ON notify.notifications (user_id) WHERE read_at IS NULL;

CREATE TABLE notify.deliveries (
  notification_id  uuid NOT NULL REFERENCES notify.notifications(id),
  channel          notify.channel NOT NULL,
  status           notify.delivery_status NOT NULL DEFAULT 'pending',
  attempts         smallint NOT NULL DEFAULT 0,
  next_attempt_at  timestamptz NOT NULL DEFAULT now(),
  provider_ref     varchar(100),
  last_error       varchar(60),                        -- an error code, never message content
  sent_at          timestamptz,
  PRIMARY KEY (notification_id, channel),
  CONSTRAINT delivery_sent_fields CHECK ((status = 'sent') = (sent_at IS NOT NULL))
);
CREATE INDEX deliveries_due ON notify.deliveries (next_attempt_at) WHERE status IN ('pending', 'failed');

-- OTPs and other synchronous SMS sent for Unit 1: no content is ever stored (SEC-DAT-07)
CREATE TABLE notify.sms_log (
  id            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  phone_hmac    bytea NOT NULL CHECK (octet_length(phone_hmac) = 32),   -- HMAC of the number, for rate
     checks
  purpose       varchar(30) NOT NULL CHECK (purpose IN ('otp', 'security_notice')),
  status        notify.delivery_status NOT NULL,
  provider_ref  varchar(100),
  simulated     boolean NOT NULL,
  at            timestamptz NOT NULL DEFAULT now()
);

-- auction watchers (FR-NTF-02)
CREATE TABLE notify.watches (
  user_id     uuid NOT NULL REFERENCES identity.users(id),
  listing_id  uuid NOT NULL REFERENCES listings.listings(id),
  created_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, listing_id)
);
CREATE INDEX watches_listing ON notify.watches (listing_id);

-- deadline reminders (FR-NTF-07): one per user, deadline kind, subject and lead time
CREATE TABLE notify.reminders (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       uuid NOT NULL REFERENCES identity.users(id),
  kind          notify.reminder_kind NOT NULL,
  subject_id    uuid NOT NULL,                          -- the order (no FK: owned by Unit 2, read via events)
  due_at        timestamptz NOT NULL,
  lead          interval NOT NULL CHECK (lead > interval '0'),
  fire_at       timestamptz NOT NULL,
  status        notify.reminder_status NOT NULL DEFAULT 'scheduled',
  sent_at       timestamptz,
  cancelled_at  timestamptz,
  CONSTRAINT reminder_once UNIQUE (user_id, kind, subject_id, lead),
  CONSTRAINT reminder_fire_time CHECK (fire_at = due_at - lead),
  CONSTRAINT reminder_fields CHECK ((status = 'sent') = (sent_at IS NOT NULL) AND (status = 'cancelled') = (
     cancelled_at IS NOT NULL))
);
CREATE INDEX reminders_due ON notify.reminders (fire_at) WHERE status = 'scheduled';

-- coalescing chat e-mails: at most one per thread per DV-41 while unread
CREATE TABLE notify.chat_email_state (
  user_id       uuid NOT NULL REFERENCES identity.users(id),
  thread_id     uuid NOT NULL REFERENCES chat.threads(id),
  last_sent_at  timestamptz NOT NULL,
  PRIMARY KEY (user_id, thread_id)
);
```

```
-- ------------------------------------------------------------------ safety of parameters (FR-NTF-08)
-- true if a text contains something no notification may carry: a phone number, an e-mail
-- address, an NID-like number, or a six-digit code
CREATE FUNCTION notify.unsafe(p text) RETURNS boolean LANGUAGE sql IMMUTABLE AS $$
  SELECT x ~ '(^|[^0-9])(880|88)?01[3-9][0-9]{8}($|[^0-9])'
      OR x ~ '[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}'
      OR x ~ '(^|[^0-9])([0-9]{10}|[0-9]{13}|[0-9]{17})($|[^0-9])'
      OR x ~ '(^|[^0-9])[0-9]{6}($|[^0-9])'
  FROM (SELECT regexp_replace(chat.normalise(p), '(?<=[0-9])[␣.\-]+(?=[0-9])', '', 'g') AS x) s
$$;
-- masks such content in user-written text (for example a listing title) before it is used
CREATE FUNCTION notify.mask(p text) RETURNS text LANGUAGE sql IMMUTABLE AS $$
  SELECT regexp_replace(regexp_replace(translate(normalize(p, NFC), '<U+09E6><U+09E7><U+09E8><U+09E9><U+09EA
     ><U+09EB><U+09EC><U+09ED><U+09EE><U+09EF>', '0123456789'),
           '[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}', '[hidden]', 'gi'),
           '\+?[0-9]([␣.\-]?[0-9]){5,}', '[hidden]', 'g')
$$;

CREATE FUNCTION notify.check_params() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE allowed text[]; k text; v jsonb;
BEGIN
  SELECT allowed_params INTO allowed FROM notify.templates WHERE code = NEW.template_code AND version = NEW.
     template_version;
  FOR k, v IN SELECT * FROM jsonb_each(NEW.params) LOOP
    IF NOT (k = ANY (allowed)) THEN
      RAISE EXCEPTION 'parameter␣%␣is␣not␣allowed␣in␣template␣%', k, NEW.template_code USING ERRCODE = '
     check_violation';
    END IF;
    IF jsonb_typeof(v) NOT IN ('string', 'number', 'boolean') THEN
      RAISE EXCEPTION 'parameter␣%␣must␣be␣a␣plain␣value', k USING ERRCODE = 'check_violation';
    END IF;
    IF jsonb_typeof(v) = 'string' AND notify.unsafe(v #>> '{}') THEN
      RAISE EXCEPTION 'FR-NTF-08:␣parameter␣%␣contains␣contact␣details␣or␣a␣code', k USING ERRCODE = '
     check_violation';
    END IF;
  END LOOP;
  RETURN NEW;
END $$;
CREATE TRIGGER notification_params BEFORE INSERT OR UPDATE OF params ON notify.notifications
  FOR EACH ROW EXECUTE FUNCTION notify.check_params();

-- FR-NTF-03 to FR-NTF-06: create a notification and its e-mail and SMS deliveries. Returns NULL
-- if the same notification was already created (an event delivered twice).
CREATE FUNCTION notify.create(p_user uuid, p_code varchar, p_params jsonb, p_dedupe varchar)
RETURNS uuid LANGUAGE plpgsql AS $$
DECLARE t notify.templates; nid uuid; email_ok boolean;
BEGIN
  SELECT * INTO t FROM notify.templates WHERE code = p_code AND active;
  IF NOT FOUND THEN RAISE EXCEPTION 'no␣active␣template␣%', p_code USING ERRCODE = 'check_violation'; END IF;
  INSERT INTO notify.notifications (user_id, template_code, template_version, params, dedupe_key)
  VALUES (p_user, t.code, t.version, coalesce(p_params, '{}'), p_dedupe)
  ON CONFLICT ON CONSTRAINT notification_once DO NOTHING RETURNING id INTO nid;
  IF nid IS NULL THEN RETURN NULL; END IF;
  SELECT coalesce((SELECT email_transactional FROM notify.preferences WHERE user_id = p_user), true) INTO
     email_ok;
  IF 'email' = ANY (t.channels) THEN
    INSERT INTO notify.deliveries (notification_id, channel, status)
    VALUES (nid, 'email', CASE WHEN t.category = 'security' OR email_ok THEN 'pending'::notify.
     delivery_status ELSE 'suppressed' END);
  END IF;
  IF 'sms' = ANY (t.channels) THEN
    INSERT INTO notify.deliveries (notification_id, channel) VALUES (nid, 'sms');
  END IF;
  RETURN nid;
END $$;

-- FR-NTF-07: schedule (or move) a reminder; a reminder already sent is re-armed only if the
-- deadline moved later (for example a takeover window restarted by a re-key)
CREATE FUNCTION notify.schedule_reminder(p_user uuid, p_kind notify.reminder_kind, p_subject uuid,
                                         p_due timestamptz, p_lead interval) RETURNS void LANGUAGE sql AS $$
  INSERT INTO notify.reminders (user_id, kind, subject_id, due_at, lead, fire_at) VALUES (p_user, p_kind,
     p_subject, p_due, p_lead, p_due - p_lead)
  ON CONFLICT ON CONSTRAINT reminder_once DO UPDATE
     SET due_at = EXCLUDED.due_at, fire_at = EXCLUDED.fire_at,
```

```
        status = CASE WHEN notify.reminders.status = 'scheduled' OR EXCLUDED.due_at > notify.reminders.
    due_at
                      THEN 'scheduled'::notify.reminder_status ELSE notify.reminders.status END,
        sent_at = CASE WHEN EXCLUDED.due_at > notify.reminders.due_at THEN NULL ELSE notify.reminders.
    sent_at END,
        cancelled_at = CASE WHEN EXCLUDED.due_at > notify.reminders.due_at THEN NULL ELSE notify.reminders.
    cancelled_at END
   WHERE notify.reminders.status <> 'cancelled' OR EXCLUDED.due_at > notify.reminders.due_at
$$;

-- the step happened: cancel the remaining reminders of that kind for the subject
CREATE FUNCTION notify.cancel_reminders(p_kind notify.reminder_kind, p_subject uuid) RETURNS integer LANGUAGE
     plpgsql AS $$
DECLARE n integer;
BEGIN
  UPDATE notify.reminders SET status = 'cancelled', cancelled_at = now()
   WHERE kind = p_kind AND subject_id = p_subject AND status = 'scheduled';
  GET DIAGNOSTICS n = ROW_COUNT; RETURN n;
END $$;

-- the reminder job claims due reminders once, even with several instances running
CREATE FUNCTION notify.claim_due_reminders(p_limit integer) RETURNS SETOF notify.reminders LANGUAGE sql AS $$
  -- a reminder whose deadline has already passed is never sent
  UPDATE notify.reminders SET status = 'cancelled', cancelled_at = now() WHERE status = 'scheduled' AND
    due_at <= now();
  UPDATE notify.reminders r SET status = 'sent', sent_at = now()
   WHERE r.id IN (SELECT id FROM notify.reminders WHERE status = 'scheduled' AND fire_at <= now() AND due_at
    > now()
                  ORDER BY fire_at LIMIT p_limit FOR UPDATE SKIP LOCKED)
  RETURNING r.*
$$;

-- DV-41: may a chat e-mail go to this user for this thread now? Records the send if so.
CREATE FUNCTION notify.chat_email_allowed(p_user uuid, p_thread uuid, p_window interval) RETURNS boolean
     LANGUAGE plpgsql AS $$
BEGIN
  INSERT INTO notify.chat_email_state (user_id, thread_id, last_sent_at) VALUES (p_user, p_thread, now())
  ON CONFLICT (user_id, thread_id) DO UPDATE SET last_sent_at = now()
   WHERE notify.chat_email_state.last_sent_at <= now() - p_window;
  RETURN FOUND;
END $$;

-- ====================================================================
-- Reference data: detection terms (FR-CHT-07, FR-GAM-20). Operators extend the list.
-- ====================================================================
INSERT INTO chat.detection_terms (term, flag) VALUES
  ('advance', 'payment_phrase'), ('send␣money', 'payment_phrase'), ('pay␣first', 'payment_phrase'),
  ('direct␣payment', 'payment_phrase'), ('personal␣number', 'payment_phrase'), ('outside␣the␣app', '
    payment_phrase'),
  ('bkash', 'payment_phrase'), ('nagad', 'payment_phrase'), ('rocket', 'payment_phrase'),
  ('<U+0985><U+0997><U+09CD><U+09B0><U+09BF><U+09AE>', 'payment_phrase'), ('<U+098F><U+09A1><U+09AD><U+09BE><
    U+09A8><U+09CD><U+09B8>', 'payment_phrase'), ('<U+0985><U+09CD><U+09AF><U+09BE><U+09A1><U+09AD><U+09BE><U
    +09A8><U+09CD><U+09B8>', 'payment_phrase'),
  ('<U+0986><U+0997><U+09C7>␣<U+099F><U+09BE><U+0995><U+09BE>', 'payment_phrase'), ('<U+09B8><U+09C7><U+09A8
    ><U+09CD><U+09A1>␣<U+09AE><U+09BE><U+09A8><U+09BF>', 'payment_phrase'), ('<U+09AC><U+09BF><U+0995><U+09BE
    ><U+09B6>', 'payment_phrase'),
  ('<U+09A8><U+0997><U+09A6><U+09C7>', 'payment_phrase'), ('<U+09B0><U+0995><U+09C7><U+099F>', '
    payment_phrase'), ('<U+09AA><U+09BE><U+09B0><U+09CD><U+09B8><U+09CB><U+09A8><U+09BE><U+09B2>␣<U+09A8><U
    +09BE><U+09AE><U+09CD><U+09AC><U+09BE><U+09B0>', 'payment_phrase'),
  ('whatsapp', 'off_platform_app'), ('telegram', 'off_platform_app'), ('imo', 'off_platform_app'),
  ('messenger', 'off_platform_app'), ('<U+09B9><U+09CB><U+09AF><U+09BC><U+09BE><U+099F><U+09B8><U+0985><U+09
    CD><U+09AF><U+09BE><U+09AA>', 'off_platform_app'), ('<U+0987><U+09AE><U+09CB>', 'off_platform_app'),
  ('password', 'credentials'), ('passcode', 'credentials'), ('pass:', 'credentials'), ('pw:', 'credentials'),
  ('login␣id', 'credentials'), ('<U+09AA><U+09BE><U+09B8><U+0993><U+09AF><U+09BC><U+09BE><U+09B0><U+09CD><U
    +09A1>', 'credentials'), ('<U+09B2><U+0997><U+0987><U+09A8>', 'credentials');

-- ====================================================================
-- Grants (SEC-INF-03)
-- ====================================================================
GRANT USAGE ON SCHEMA chat, notify TO tb_app;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA chat, notify TO tb_app;
GRANT DELETE ON chat.blocks, notify.watches TO tb_app;                          -- unblock, unwatch
GRANT DELETE ON notify.notifications, notify.deliveries, notify.sms_log TO tb_app;   -- retention jobs (DV
    -42)
REVOKE UPDATE ON chat.staff_access_log, notify.sms_log FROM tb_app;
```